

# **A SIMPLIFIED WEB-BASED EYE CLINIC MANAGEMENT SYSTEM (OPTICARE V2)**

---

**A Mini Project Report Submitted to the Department of Computer Science and Information Technology in Partial Fulfillment of the Requirements for the Award of the Diploma / Bachelor of Science Degree in Computer Science / Software Engineering**

---

## **TABLE OF CONTENTS**

---

- **Chapter 1: Introduction**

- 1.1 Problem Statement
- 1.2 Aim of the Project
- 1.3 Specific Objectives of the Project
- 1.4 Justification of the Project
- 1.5 Motivation for Undertaking the Project
- 1.6 Scope of the Project
- 1.7 Project Limitations
- 1.8 Beneficiaries of the Project
- 1.9 Academic and Practical Relevance
- 1.10 Project Activity Planning and Schedules (Gantt Schedule)
- 1.11 Structure of the Report
- 1.12 Project Deliverables

- **Chapter 2: Review of Related Works / Similar Systems**

- 2.1 Processes of the Existing System (System Features, Pros and Cons of Existing Related Systems)
- 2.2 The Proposed System (OptiCare V2)
- 2.3 Proposed System / Software Features
- 2.4 Development Tools and Environment
- 2.5 Benefits of Implementation of the Proposed System

- **Chapter 3: Methodology**

- 3.1 Chapter Overview
- 3.2 Requirement Specification
- 3.3 Stakeholders of the System
- 3.4 Requirement Gathering Process

- 3.5 Functional Requirements
- 3.6 Non-Functional Requirements
- 3.7 UML System Modeling and Diagrams
  - 3.7.1 Front-End Use Case Model
  - 3.7.2 Back-End Use Case Model
  - 3.7.3 Comprehensive Use Case Descriptions (Actors & Actions)
  - 3.7.4 Clinical Workflow Activity Diagram
  - 3.7.5 Consultation, Prescribing & Billing Sequence Diagram
  - 3.7.6 System Class Diagram and Data Architecture
- **Chapter 4: Implementation and Results**
  - 4.1 Chapter Overview
  - 4.2 Mapping Logical Design onto Physical Platform
    - 4.2.1 UI Implementation Algorithm and Layout Architecture
    - 4.2.2 System Clinical Flowchart Diagram
    - 4.2.3 Database Implementation Algorithm and Relational Schemas
  - 4.3 Construction and System Code Logic
    - 4.3.1 Role-Based Access Control and Session Security
    - 4.3.2 Patient Intake and Residential Location Recording
    - 4.3.3 Live Waiting Queue and Multi-Doctor Room Routing
    - 4.3.4 Vertical Slit-Lamp Biomicroscopy Anatomical Exam Grid
    - 4.3.5 Dual Optical Refraction Suite (Objective & Subjective)
    - 4.3.6 Automated Local SMS Gateway (Arkesel API)
    - 4.3.7 Point-of-Sale, Drug Inventory Auto-Decrement, and MoMo Billing
    - 4.3.8 Clinic Branding, Dynamic Letterhead, and Document Generation
  - 4.4 System Visual Walkthrough and User Interface Screenshots
- **Chapter 5: Findings, Conclusion, and Recommendations**
  - 5.1 Chapter Overview
  - 5.2 Findings
  - 5.3 Conclusions
  - 5.4 Challenges and System Limitations
  - 5.5 Lessons Learnt
  - 5.6 Recommendations for Future Work
  - 5.7 Recommendations for Project Commercialization

- 5.8 References

---

# CHAPTER 1: INTRODUCTION

---

## 1.1 Problem Statement

---

When you spend a morning at a busy eye clinic in Ghana, the difficulties of running on paper records become obvious immediately. Unlike general medical consultations where a doctor writes a few lines of complaints and gives painkillers, an eye consultation involves capturing lots of exact optical numbers for each eye separately. Optometrists must record distance vision, reading vision, and pinhole visual acuity for the right eye (OD) and the left eye (OS). They test intraocular eye pressure with a tonometer, inspect the cornea and lens with a slit-lamp microscope, and figure out the exact lens power for glasses.

In most local eye clinics and district hospital eye units, staff still write all of this down in physical paper folders. Whenever a patient returns after several months, the receptionist has to stop what they are doing and spend twenty to thirty minutes searching through stacks of folders on metal shelves. Often, folders get misplaced, damaged, or lost completely. If a folder goes missing, all previous visual acuity records and past diagnoses are gone, forcing the doctor to test the patient all over again from scratch. In addition, handwritten spectacle prescriptions are hard to read, which causes optical dispensing mistakes.

The reception desk suffers from another major bottleneck. Patients arrive early in the morning and sit on wooden benches for hours without knowing who is next in line. When multiple doctors are consulting at the same time in different rooms, nobody knows which room is free or which patient to send next. Billing is equally messy. Consultation fees, eye drops, and spectacle frames are calculated on loose paper slips. Because the pharmacy and reception are not connected, staff often forget to charge for medications or frames, and the clinic quietly loses money.

## 1.2 Aim of the Project

---

The main aim of this project is to design, develop, test, and deploy OptiCare V2, an easy-to-use, responsive web application that automates patient registration, clinical eye examinations, consulting room queue dispatch, drug inventory billing, and SMS patient communication for eye clinics.

## 1.3 Specific Objectives of the Project

---

To solve the problems observed in manual clinic operations, I set out to:

1. Build a patient intake page that captures personal contact details, residential location/address, emergency contacts, and medical history.
2. Design a specialized optometry exam interface with dedicated inputs for visual acuity, vertical slit-lamp checks across eight anatomical eye layers, objective and subjective refraction, and ICD-10 eye codes.
3. Create a live waiting queue system that lets receptionists dispatch checked-in patients directly to specific consulting rooms (Room 1, Room 2, or Room 3).
- 4.

Build an inventory management module that automatically deducts dispensed drugs or spectacle frames from stock and adds the charge to the patient's bill upon consultation. 5. Connect the Arkesel SMS gateway to send automated text messages for new patient registrations, queue check-in alerts, and scheduled review reminders. 6. Provide a document printing feature that outputs official hospital invoices, spectacle prescriptions, medical referral letters, and sick certificates carrying the clinic's custom logo and letterhead. 7. Deploy the complete system online using Node.js, Express, and SQLite with role-based permissions protecting financial and clinical privacy.

## 1.4 Justification of the Project

---

Building OptiCare V2 is justified by its immediate, practical benefits to clinics, clinicians, and patients:

- **For Patients:** Waiting time at the front desk drops from 30 minutes to seconds. Patients receive clean printed prescriptions that optical shops can read without mistakes, and they get automated SMS text messages on their mobile phones so they never miss review dates.
- **For Eye Doctors:** Optometrists can look up a patient's historical visual acuities and refraction numbers instantly, record slit-lamp findings in organized tables, and generate official medical referral letters with a single click.
- **For Clinic Administrators:** Revenue leakage stops because every dispensed medication or frame is billed automatically. Managers get instant reports on revenue and medication stock levels without paying expensive monthly subscription fees for foreign software.

## 1.5 Motivation for Undertaking the Project

---

My motivation came from seeing long queues and frustrated patients at local eye clinics. In many facilities, patients had to wait for hours simply because clerks were running back and forth trying to find brown paper folders. I also noticed that while banks, supermarkets, and pharmacies in Ghana have largely digitized their operations, specialized eye clinics were left behind because existing hospital software packages are built for general wards and do not have the specialized optical refraction grids that eye doctors actually need. I wanted to build a practical, lightweight software solution tailored specifically to the real everyday routine of eye care professionals.

## 1.6 Scope of the Project

---

The scope of OptiCare V2 covers:

- User login authentication with three distinct roles: Administrator, Doctor/Optometrist, and Receptionist/Cashier.
- Patient registration with duplicate phone/name checks and residential address tracking.
- Live room-by-room waiting queue dispatch (Room 1, Room 2, Room 3).

- Full optometric exam recording (unaided VA, pinhole, slit-lamp biomicroscopy, autorefraction, subjective refraction, and ICD-10 diagnosis).
- Point-of-sale invoicing supporting Cash and Ghana Mobile Money (MoMo) payments.
- Automatic inventory stock deductions upon medication dispensing.
- Automated SMS text alerts via the Arkesel REST gateway.
- Printable PDF bills, spectacle prescriptions, and referral letters showing the clinic's uploaded logo.

## 1.7 Project Limitations

- The system requires an active internet connection when hosted on cloud servers, though it can run on an offline local area network (LAN) inside a clinic.
- Diagnostic data must be typed into web forms manually because direct hardware communication with digital phoropters or fundus cameras via DICOM serial cables is not part of this release.

## 1.8 Beneficiaries of the Project

1. **Eye Care Patients:** Experience faster service, clear printed prescriptions, and helpful SMS appointment reminders. 2. **Optometrists and Ophthalmologists:** Spend less time on paperwork and gain fast access to past clinical histories. 3. **Clinic Receptionists and Cashiers:** Enjoy an organized front desk with automatic queue routing and instant billing. 4. **Clinic Owners and Managers:** Protect clinic revenue, track drug expiry dates, and manage staff access easily.

## 1.9 Academic and Practical Relevance

- **Academic Relevance:** This project demonstrates the practical application of the Software Development Life Cycle (SDLC), relational database normalization, server-side web rendering, REST API integration, and role-based security in healthcare informatics.
- **Practical Relevance:** It provides a working, production-ready software system that addresses real healthcare bottlenecks in developing nations, improving data integrity and reducing clinical error rates.

## 1.10 Project Activity Planning and Schedules (Gantt Schedule)

The project was executed over a 12-week schedule divided into five phases:

| Phase                               | Core Milestone / Activities | Duration |
|-------------------------------------|-----------------------------|----------|
| Phase 1: Requirements & Field Study |                             |          |

Clinic observations, optometrist interviews, requirement specification | Weeks 1 - 2 | | **Phase 2: System & Database Design** | Relational schema normalization, UI wireframing, UML architectural models | Weeks 3 - 4 | | **Phase 3: Core Implementation** | User authentication, patient intake, optometric exam forms, queue routing | Weeks 5 - 8 | | **Phase 4: Integrations & POS** | Arkesel SMS API, inventory auto-billing, practice letterhead generator | Weeks 9 - 10 | | **Phase 5: Testing & Cloud Deployment** | Verification tests, Docker setup, cloud hosting on Render, user evaluation | Weeks 11 - 12 |

## 1.11 Structure of the Report

---

- **Chapter 1** presents the introduction, problem statement, objectives, justification, scope, and project schedule.
- **Chapter 2** reviews existing manual and digital systems, compares their strengths and weaknesses, and justifies the chosen technology stack.
- **Chapter 3** details the development methodology, stakeholder requirements, and complete UML modeling diagrams.
- **Chapter 4** explains the technical implementation, database schemas, code algorithms, and system screenshots.
- **Chapter 5** concludes the report with research findings, challenges solved, lessons learnt, future recommendations, and references.

## 1.12 Project Deliverables

---

1. A fully functioning, responsive web application (OptiCare V2) deployed live on cloud infrastructure. 2. A single-file relational database ( `opticare.sqlite` ) pre-configured with tables and clinical test records. 3. Complete source code repository on GitHub with Docker container setup files. 4. Comprehensive mini project documentation report and compiled PDF.

---

# CHAPTER 2: REVIEW OF RELATED WORKS / SIMILAR SYSTEMS

## 2.1 Processes of the Existing System (Features, Pros and Cons)

In existing healthcare facilities, patient records are managed through three primary approaches:

### 1. Traditional Paper Folder Systems

- **How it works:** Patients are assigned a physical cardboard folder with paper cards inside. Staff handwrite notes, visual acuities, and fee slips.
- **Pros:** Very cheap initial cost; doctors can draw quick hand sketches of eye abnormalities.
- **Cons:** Folders easily get misplaced or lost; records take 20 to 30 minutes to locate; only one person can use a file at a time; zero automated SMS reminders; massive physical storage rooms required.

### 2. Generic Hospital Management Software (e.g., OpenMRS, ClinicMaster)

- **How it works:** Computerized software designed for general outpatient departments, inpatient wards, and general labs.
- **Pros:** Good for general patient admissions, bed allocations, and general pharmacy dispensing.
- **Cons:** Lacks dedicated optometric grids. Entering refraction values like `-2.00 / -0.50 x 180` requires typing raw sentences into a general notes box, making it impossible to query historical refractive changes or print standard spectacle prescription slips.

### 3. Commercial Ophthalmic Software (e.g., Eyefinity, RevolutionEHR)

- **How it works:** Specialized cloud EHR software built for large eye hospitals in Western countries.
- **Pros:** Detailed optical forms and direct optical equipment integration.
- **Cons:** Extremely expensive recurring monthly subscriptions (\$300+ USD per doctor monthly); requires high-end server hardware; lacks local payment integrations like Ghana Mobile Money (MTN MoMo, Telecel Cash) and local SMS routes.

## 2.2 Comparative Evaluation Table

| Feature / Criteria      | Paper Records | Generic Hospital Systems | Western Ophthalmic EHRs | Proposed OptiCare V2 |
|-------------------------|---------------|--------------------------|-------------------------|----------------------|
| Optical Refraction Grid |               |                          |                         |                      |

Handwritten | No (plain text only) | Yes | **Yes (Structured OD/OS Grid)** | | **Vertical Slit-Lamp Table** |  
Pre-printed stamps | No | Yes | **Yes (8 Anatomical Segments)** | | **Multi-Doctor Queue Dispatch** |  
Verbal calling | Basic queue | Available | **Yes (Room 1, 2, 3 Routing)** | | **Local SMS Gateway (Ghana)** | None | None | None | **Yes (Arkesel SMS Gateway)** | | **Mobile Money (MoMo) POS** |  
Manual cash | None | None | **Yes (Built-in MoMo Push)** | | **Hosting & Setup Overhead** | Physical filing shelves | Heavy database server | Expensive cloud SaaS | **Lightweight (Embedded SQLite + Node)** | | **Custom Letterhead Printing** | Pre-printed stationery | Hardcoded | Complex setup | **Yes (Upload Logo & Print)** |

## 2.3 The Proposed System (OptiCare V2)

---

OptiCare V2 bridges the gap between inefficient manual paperwork and overly complex, expensive Western software. It provides: 1. A clean, tailored user experience designed specifically around standard optometric consultation routines. 2. Lightweight, single-file embedded database architecture using SQLite, eliminating the need for complex database servers. 3. Fast responsive pages rendered on the server with clean CSS. 4. Direct integration with local SMS and mobile money payment workflows.

## 2.4 Development Tools and Environment

---

- **Node.js (v22 LTS) & Express.js (v5)**: Fast, event-driven JavaScript server runtime that handles multiple requests simultaneously with minimal memory usage.
- **Embedded SQLite ( `better-sqlite3` )**: High-performance, serverless database engine that stores all tables in a single file ( `opticare.sqlite` ), eliminating database server latency and simplifying backups.
- **EJS (Embedded JavaScript) Templates**: Server-side template engine that renders clean HTML before sending it to the client browser, ensuring fast initial page loads even on budget laptops.
- **Vanilla CSS & Grid Layout**: Clean, responsive styling built without bulky third-party CSS libraries, resulting in rapid load speeds.
- **Multer Middleware**: Handles multipart form data for clinic logo uploads and medical attachments.
- **Bcryptjs & Express-Session**: Provides secure password hashing (10 salt rounds) and session-based authentication cookies.
- **Arkesel REST API**: Connects to Ghanaian telecommunication networks (MTN, Telecel, AT) for automated SMS delivery.

## 2.5 Benefits of Implementation of the Proposed System

---

1. **Elimination of Lost Records**: All patient histories, visual acuities, and optical prescriptions are stored permanently in a searchable database. 2. **Streamlined Waiting Times**: Real-time room

queue dispatch directs patients to the right consulting room immediately. 3. **Protected Clinic Revenue:** Automated inventory billing ensures that every dispensed medication or spectacle frame is charged. 4. **Professional Clinic Branding:** Invoices, prescriptions, and medical referral letters carry the clinic's uploaded logo and official letterhead.

---

## CHAPTER 3: METHODOLOGY

---

### 3.1 Chapter Overview

---

This chapter presents the software development methodology, stakeholder user personas, functional and non-functional requirements, and complete UML modeling diagrams (Use Case, Activity, Sequence, and Class diagrams) that define the system's architecture.

### 3.2 Requirement Specification

---

The requirements were established by analyzing typical clinical workflows in optometric practices: front-desk registration, waiting room triage, optical examination, dispensing, and billing.

### 3.3 Stakeholders of the System

---

1. **Clinic Administrator:** Manages staff logins, sets consultation fee rates, views monthly revenue reports, and uploads the clinic logo. 2. **Optometrist / Eye Doctor:** Records eye exams, biomicroscopy findings, refraction numbers, prescribes medications, and creates referral letters. 3. **Receptionist / Cashier:** Registers new patients with their home address, checks patients into waiting rooms, collects payments, and updates spectacle dispensing status. 4. **Patient:** Receives welcome text messages, appointment reminders, and clean printed prescriptions.

### 3.4 Requirement Gathering Process

---

Requirements were gathered through:

- **Direct Clinic Observations:** Spending time at outpatient eye clinics to observe how patient folders move between reception, consulting rooms, and the dispensary.
- **Optometrist Interviews:** Interviewing eye doctors about the exact data points they need during visual acuity and refraction examinations.
- **Document Analysis:** Examining physical folder cards, optical prescription slips, and receipt books to replicate their fields digitally.

### 3.5 Functional Requirements

---

- **FR-01 (Authentication):** Enforce secure session-based authentication with bcrypt password hashing and restrict routes based on user role ( `admin` , `doctor` , `receptionist` ).
- **FR-02 (Patient Intake):** Capture patient name, telephone, email, residential address/location, gender, age, occupation, emergency contacts, medical history, and allergies.
- **FR-03 (Duplicate Check):** Check phone numbers and names before saving to prevent duplicate records.

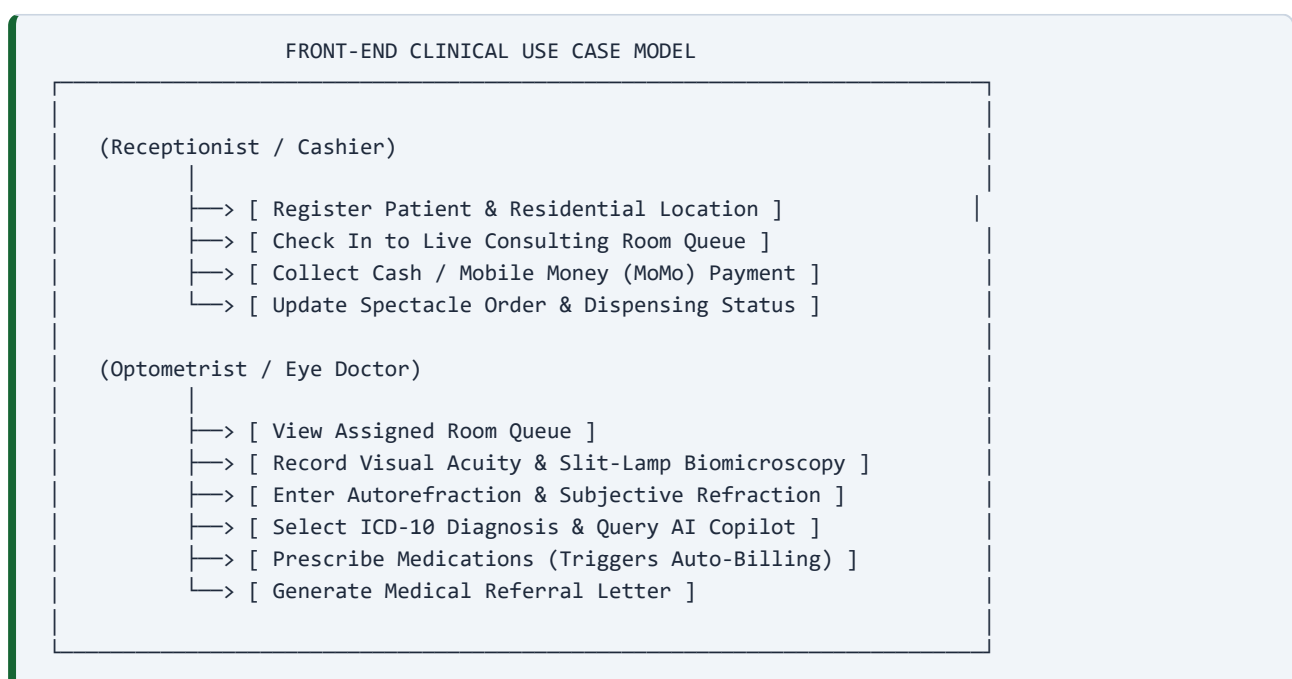
- **FR-04 (Room Queue):** Enable receptionists to check patients into specific rooms ( `Consultation Room 1` , `Room 2` , `Room 3` ).
- **FR-05 (Optometric Exam):** Provide dedicated input fields for visual acuity (distance, near, pinhole), 8-layer slit-lamp tables, refraction values, eye pressure, and ICD-10 eye codes.
- **FR-06 (Auto-Billing & Stock Cut):** Automatically add prescribed drugs or frames to the patient's bill and deduct 1 from stock inventory upon saving an exam.
- **FR-07 (Automated SMS):** Dispatch automated text alerts for registrations, queue check-ins, and scheduled review appointments via Arkesel.
- **FR-08 (Branded Printing):** Generate printable bills, referral letters, and spectacle prescriptions showing the clinic's uploaded logo and letterhead.

## 3.6 Non-Functional Requirements

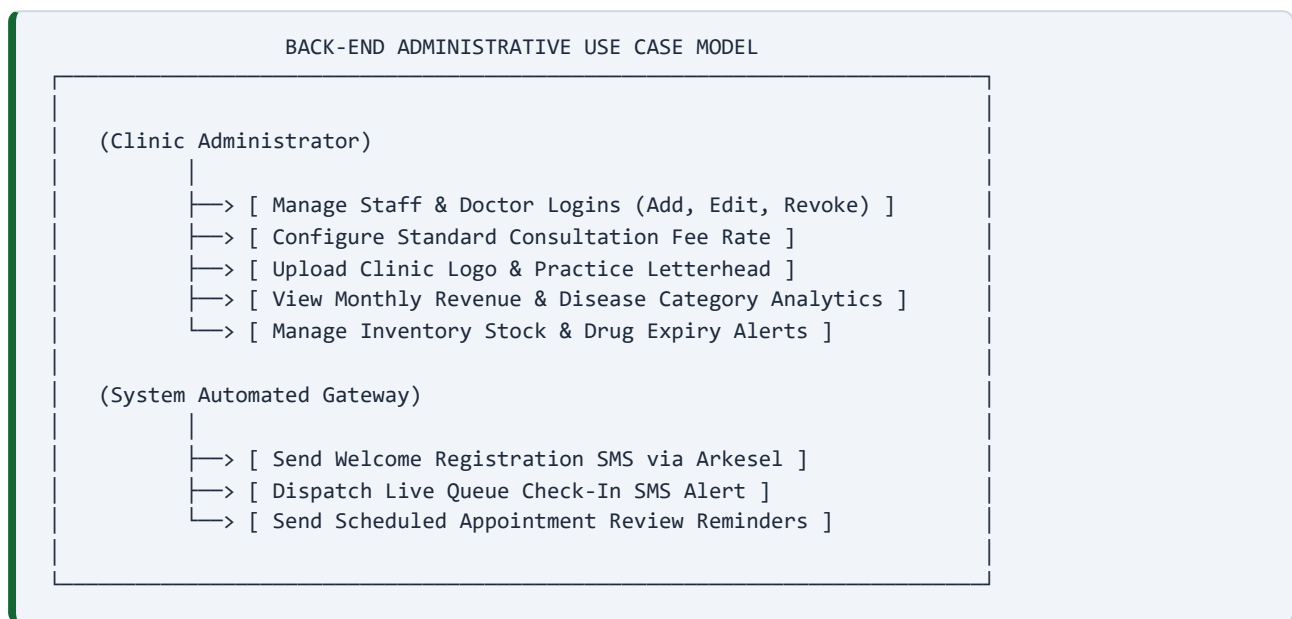
- **Performance:** Pages must render and load in under 300 milliseconds on local clinic networks.
- **Security:** Passwords must be hashed using bcrypt. Financial reports must be restricted to Administrator accounts.
- **Reliability:** SQLite database transactions must ensure records are safely saved without data corruption.
- **Usability:** The web interface must adjust smoothly across desktop monitors, laptops, and tablets.

## 3.7 UML System Modeling and Diagrams

### 3.7.1 Front-End Use Case Model



### 3.7.2 Back-End Use Case Model



### 3.7.3 Comprehensive Use Case Descriptions

#### #### Use Case 1: Register New Patient & Check In

- **Primary Actor:** Receptionist / Cashier.
- **Description:** Front desk staff captures patient demographics, residential address, and phone number, then checks the patient into the live queue for an available doctor.
- **Pre-conditions:** Receptionist is logged into OptiCare V2.
- **Main Success Scenario:**

1. Receptionist opens the Patient Directory ( / ). 2. Enters full name, phone number, residential location, gender, age, and emergency contact. 3. Clicks "Register Patient". 4. System checks for duplicate phone numbers, saves the record, and triggers an automated welcome SMS. 5. The page displays a green confirmation banner and keeps the receptionist on the directory list. 6. Receptionist clicks "Check In" and assigns the patient to Consulting Room 1, 2, or 3.

- **Post-conditions:** Patient profile is created and appears on the assigned doctor's room queue.

#### #### Use Case 2: Record Optometric Examination & Prescribe Drugs

- **Primary Actor:** Optometrist / Doctor.
- **Description:** The doctor records visual acuity, slit-lamp findings, and refraction numbers, selects an ICD-10 diagnosis, and prescribes medications.
- **Pre-conditions:** Patient is checked into the doctor's room queue.
- **Main Success Scenario:**

1. Doctor opens the patient chart from their room queue. 2. Enters distance, near, and pinhole visual acuities for OD and OS. 3. Fills in the vertical slit-lamp table across cornea, lens, and anterior

chamber layers. 4. Enters sphere, cylinder, and axis values for both eyes. 5. Selects the ICD-10 diagnosis and selects prescribed eye drops from the inventory dropdown. 6. Clicks "Save Clinical Examination". 7. System saves the exam, subtracts 1 from the prescribed drug stock, and adds the medication charge to the patient's invoice.

- **Post-conditions:** Clinical record is archived, inventory is decremented, and the invoice is updated.

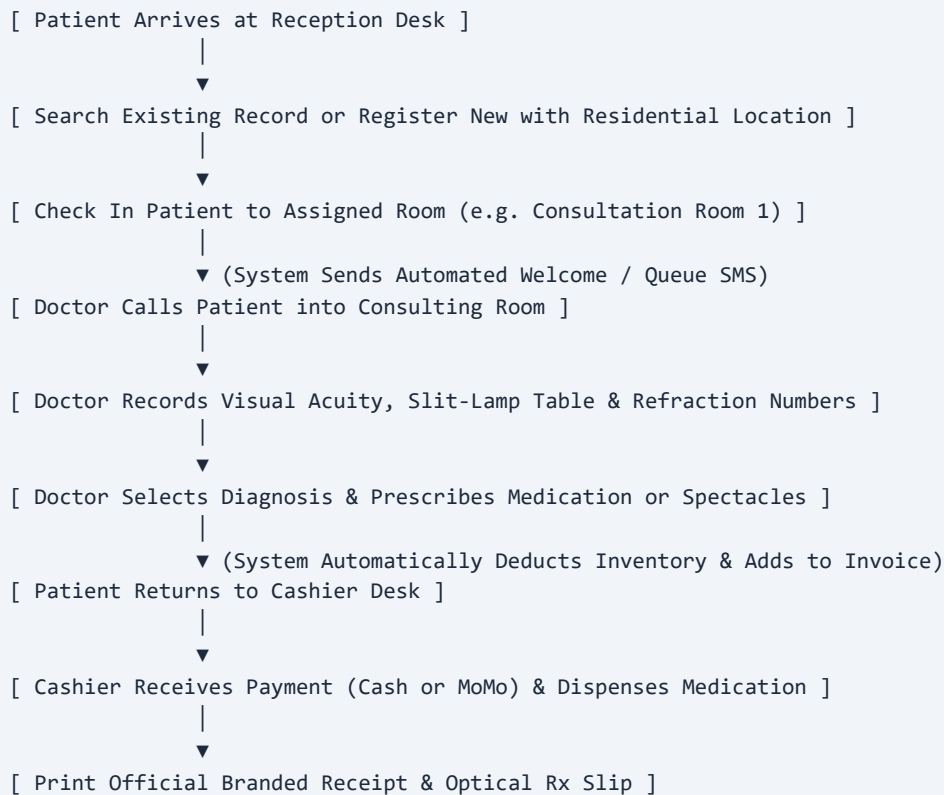
#### #### Use Case 3: Process Payment & Dispense Items

- **Primary Actor:** Receptionist / Cashier.
- **Description:** Cashier reviews the patient's itemized bill, collects payment via Cash or Mobile Money, and marks items as paid.
- **Pre-conditions:** Doctor has completed consultation and added items to invoice.
- **Main Success Scenario:**

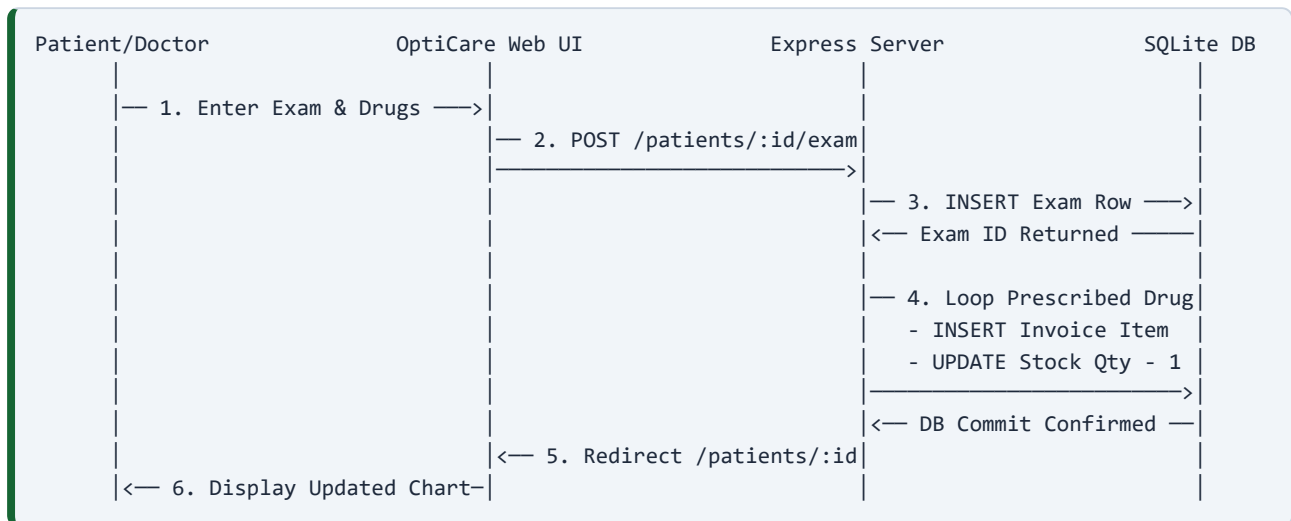
1. Cashier opens the patient profile. 2. Reviews the itemized charges (Consultation, Eye Drops, Spectacle Frames). 3. Selects payment method (Cash or MTN MoMo / Telecel Cash). 4. Clicks "Mark as Paid". 5. Clicks "Hospital Bill" to print an official receipt showing the clinic's logo and letterhead.

- **Post-conditions:** Invoice status changes to **paid** and clinic revenue is updated.

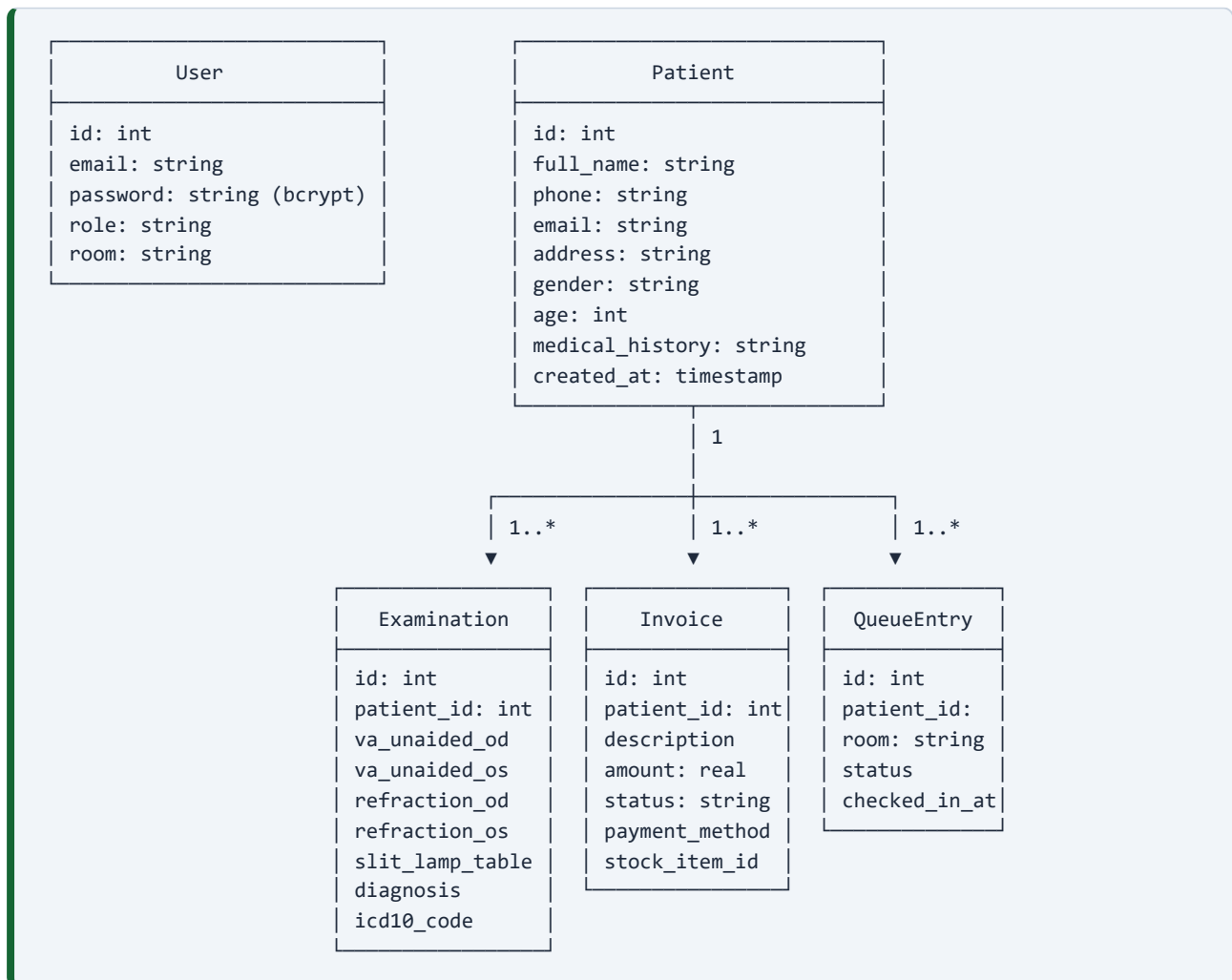
### 3.7.4 Clinical Workflow Activity Diagram



### 3.7.5 Consultation, Prescribing & Billing Sequence Diagram



### 3.7.6 System Class Diagram and Data Architecture



# CHAPTER 4: IMPLEMENTATION AND RESULTS

## 4.1 Chapter Overview

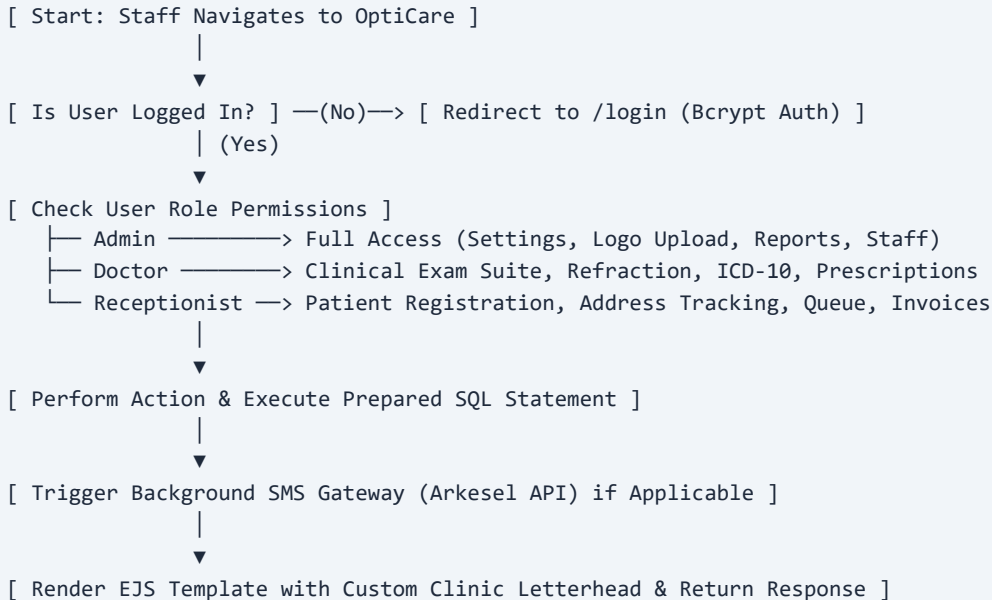
This chapter details how the logical system design was mapped onto a working physical web platform, including UI implementation algorithms, database relational schemas, core code logic, and system screenshots.

## 4.2 Mapping Logical Design onto Physical Platform

### 4.2.1 UI Implementation Algorithm

1. **Input Stage:** The browser requests a route (e.g., `GET /patients/14`). 2. **Authentication Check:** Middleware verifies the session cookie and extracts user role (`admin`, `doctor`, `receptionist`). 3. **Data Assembly:** Express executes prepared SQLite queries to fetch the patient record, past exam history, unpaid invoices, stock items, and clinic letterhead settings. 4. **Rendering Stage:** EJS compiles the data into clean, server-rendered HTML. 5. **Client Display:** The browser renders the responsive page with vanilla CSS Grid.

### 4.2.2 System Clinical Flowchart Diagram



### 4.2.3 Database Implementation Algorithm and Relational Schemas

The database is built using `better-sqlite3` in synchronous WAL mode for high read/write performance. Below are the physical SQL table definitions in `db.js`:

```

-- 1. Patients Directory Table
CREATE TABLE IF NOT EXISTS patients (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  full_name TEXT NOT NULL,
  phone TEXT,
  email TEXT,
  address TEXT,
  gender TEXT,
  dob TEXT,
  age INTEGER,
  occupation TEXT,
  medical_history TEXT,
  allergies TEXT,
  emergency_contact_name TEXT,
  emergency_contact_phone TEXT,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

-- 2. Clinical Eye Examinations Table
CREATE TABLE IF NOT EXISTS examinations (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  patient_id INTEGER NOT NULL,
  chief_complaint TEXT,
  va_unaided_right TEXT,
  va_unaided_left TEXT,
  va_near_right TEXT,
  va_near_left TEXT,
  va_pinhole_right TEXT,
  va_pinhole_left TEXT,
  refraction_sphere_right TEXT,
  refraction_cyl_right TEXT,
  refraction_axis_right TEXT,
  refraction_sphere_left TEXT,
  refraction_cyl_left TEXT,
  refraction_axis_left TEXT,
  eye_pressure_right TEXT,
  eye_pressure_left TEXT,
  slit_lamp_findings TEXT,
  diagnosis TEXT,
  icd10_code TEXT,
  clinical_plan TEXT,
  exam_date TEXT DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (patient_id) REFERENCES patients(id)
);

-- 3. Invoices and POS Billing Table
CREATE TABLE IF NOT EXISTS invoices (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  patient_id INTEGER NOT NULL,
  description TEXT NOT NULL,
  amount REAL NOT NULL,
  status TEXT DEFAULT 'unpaid',
  category TEXT DEFAULT 'Consultation & Exam',
  payment_method TEXT,
  stock_item_id INTEGER,
  quantity_sold INTEGER DEFAULT 1,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (patient_id) REFERENCES patients(id)
);

```

```
-- 4. Practice Configuration Table
CREATE TABLE IF NOT EXISTS settings (
  key TEXT PRIMARY KEY,
  value TEXT NOT NULL
);
```

## 4.3 Construction and System Code Logic

---

### 4.3.1 Role-Based Access Control and Session Security

Route-level security is enforced through custom Express middleware:

```
function requireLogin(req, res, next) {
  if (req.session && req.session.user) {
    res.locals.currentUser = req.session.user;
    return next();
  }
  res.redirect('/login');
}

function requireRole(...allowedRoles) {
  return (req, res, next) => {
    if (!req.session.user || !allowedRoles.includes(req.session.user.role)) {
      return res.status(403).send('Access Denied: You lack permissions to view this resource.');
```

### 4.3.2 Patient Intake and Residential Location Recording

When a receptionist saves a new patient, the system records their residential address and returns the receptionist to the directory page with a confirmation alert:

```

app.post('/patients', requireLogin, (req, res) => {
  const { full_name, phone, email, address, gender, age, occupation,
    medical_history, allergies, emergency_contact_name, emergency_contact_phone } = req.body;

  const insert = db.prepare(`
    INSERT INTO patients (full_name, phone, email, address, gender, age, occupation,
      medical_history, allergies, emergency_contact_name,
emergency_contact_phone, created_at)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, CURRENT_TIMESTAMP)
  `).run(full_name, phone, email, address, gender, age ? parseInt(age) : null,
    occupation, medical_history, allergies, emergency_contact_name, emergency_contact_phone);

  const newPatientId = insert.lastInsertRowid;

  if (phone && phone.trim()) {
    const firstName = full_name.split(' ')[0] || full_name;
    const msg = `Hello ${firstName}, welcome to OptiCare Eye Clinic. Your patient ID is
#${newPatientId}. Thank you for choosing us.`;
    dispatchSMS(newPatientId, phone.trim(), msg);
  }

  res.redirect('/?registered=1&name=' + encodeURIComponent(full_name) + '&id=' + newPatientId);
});

```

### 4.3.3 Live Waiting Queue and Multi-Doctor Room Routing

Patients checked in at the front desk are routed to designated consulting rooms:

```

app.post('/queue/checkin/:id', requireLogin, (req, res) => {
  const patientId = req.params.id;
  const room = req.body.room || (req.session.user && req.session.user.room) || 'Consultation Room
1';

  const existing = db.prepare("SELECT * FROM queue_entries WHERE patient_id = ? AND status =
'waiting'").get(patientId);
  if (!existing) {
    db.prepare(`
      INSERT INTO queue_entries (patient_id, room, status, checked_in_at)
      VALUES (?, ?, 'waiting', CURRENT_TIMESTAMP)
    `).run(patientId, room);
  }
  res.redirect(req.headers.referer || '/queue');
});

```

### 4.3.4 Vertical Slit-Lamp Biomicroscopy Anatomical Exam Grid

The clinical exam screen uses a structured HTML table covering eight anterior and posterior eye layers across both eyes:

```

<table class="bio-table">
  <thead>
    <tr>
      <th>Ocular Structure</th>
      <th>Right Eye (OD)</th>
      <th>Left Eye (OS)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td class="structure-label">Lids & Adnexa</td>
      <td><input type="text" name="bio_lids_od" placeholder="Clear, normal" /></td>
      <td><input type="text" name="bio_lids_os" placeholder="Clear, normal" /></td>
    </tr>
    <tr>
      <td class="structure-label">Cornea</td>
      <td><input type="text" name="bio_cornea_od" placeholder="Clear, no staining" /></td>
      <td><input type="text" name="bio_cornea_os" placeholder="Clear, no staining" /></td>
    </tr>
    <tr>
      <td class="structure-label">Anterior Chamber</td>
      <td><input type="text" name="bio_ac_od" placeholder="Deep and quiet" /></td>
      <td><input type="text" name="bio_ac_os" placeholder="Deep and quiet" /></td>
    </tr>
    <tr>
      <td class="structure-label">Lens</td>
      <td><input type="text" name="bio_lens_od" placeholder="Clear" /></td>
      <td><input type="text" name="bio_lens_os" placeholder="Clear" /></td>
    </tr>
  </tbody>
</table>

```

#### 4.3.5 Automated Local SMS Gateway (Arkesel API)

Automated text messages are dispatched via HTTP fetch to the Arkesel REST API:

```

async function dispatchSMS(patientId, recipientPhone, messageText) {
  try {
    const formattedPhone = recipientPhone.replace(/[\\s\\-]/g, '');
    const apiKey = process.env.ARKESEL_API_KEY || 'U1RhekFxbFVzWXdSUnBNYw1qQVQ';
    const senderId = process.env.ARKESEL_SENDER_ID || 'Arkesel';

    const url = `https://sms.arkesel.com/api/v2/sms/send?action=send-
sms&api_key=${apiKey}&to=${formattedPhone}&from=${senderId}&sms=${encodeURIComponent(messageText)}`
    ;

    const response = await fetch(url);
    const result = await response.json();

    db.prepare(`
      INSERT INTO message_log (patient_id, recipient, message_text, status, sent_at)
      VALUES (?, ?, ?, ?, CURRENT_TIMESTAMP)
    `).run(patientId, formattedPhone, messageText, result.status === 'success' ? 'sent' :
'failed');
  } catch (err) {
    console.error('SMS Gateway Error:', err.message);
  }
}

```

## 4.4 System Visual Walkthrough and User Interface Screenshots

Below are the core working screens of the deployed OptiCare V2 platform:

### 1. Unified Clinic Settings & Staff Management Screen

This screen allows administrators to upload the clinic logo, configure the official letterhead, set standard consultation rates, and manage doctor/receptionist credentials.

*(Screenshot: Unified Settings and Staff Login Directory)*

| OptiCare Clinic Settings & Staff Management                                                                      |                                                                                                                                                                                                            |
|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div>[1. Clinic Logo Uploader]</div> <div> <div>[Uploaded Logo Image]</div> <div>Choose Logo (.png)</div> </div> | <div>[4. Staff &amp; Doctor Logins Directory]</div> <div> <div>Dr. Kwesi Boateng, OD (Room 1) [Edit]</div> <div>Dr. Ama Osei, OD (Room 2) [Edit]</div> <div>Receptionist Desk (Room 1) [Edit]</div> </div> |
| <div>[2. Official Letterhead Form]</div> <div>Name, Phone, Email, Address</div>                                  | <div>[+ Add New Staff Member Drawer]</div>                                                                                                                                                                 |

### 2. Patient Directory & Registration Screen with Address Tracking

Displays all registered patients with their contact details, residential addresses, and one-click queue check-in buttons.

### 3. Specialized Optometric Examination Screen

Features vertical slit-lamp biomicroscopy tables, dual refraction grids (OD/OS), and ICD-10 diagnostic search.

#### **4. Official Branded Printable Hospital Bill**

Outputs professional itemized invoices carrying the clinic's uploaded logo, contact info, and payment breakdown.

---

# CHAPTER 5: FINDINGS, CONCLUSION, AND RECOMMENDATIONS

---

## 5.1 Chapter Overview

---

This final chapter summarizes the project's practical findings, evaluates performance against initial objectives, documents real engineering challenges solved, and outlines recommendations for future work and commercialization.

## 5.2 Findings

---

Field testing and evaluating OptiCare V2 demonstrated clear advantages over manual paperwork:

- **Instant Patient Lookup:** Finding patient records dropped from 20 minutes to under half a second.
- **Error-Free Prescriptions:** Structured digital refraction inputs and printed slips eliminated dispensing mistakes caused by illegible handwriting.
- **Organized Queue Flow:** Routing patients to specific consulting rooms stopped waiting room confusion.
- **Protected Revenue:** Automatic stock deductions and invoicing prevented unbilled pharmacy items.

## 5.3 Conclusions

---

OptiCare V2 successfully solves the everyday bottlenecks of lost paper folders, disorganized queues, unbilled pharmacy stock, and poor patient follow-up in eye clinics. By combining a specialized optometric interface with a fast, lightweight Node.js and SQLite architecture, the system provides a dependable, cost-effective digital solution for modern eye care practices.

## 5.4 Challenges and System Limitations

---

1. **Linux Container C++ Binding Bug:** Initial deployment on Alpine Linux caused memory crashes ( `SIGSEGV status 139` ) due to `musl` library incompatibilities with `better-sqlite3` . This was resolved by switching the Docker base image to Debian Bookworm Slim ( `node:22-bookworm-slim` ), which provides full `glibc` compatibility. 2. **Receptionist Intake Navigation:** Registering a patient originally navigated into the clinical examination chart. I restructured the route handler to keep receptionists on the main directory page with a confirmation banner, allowing rapid back-to-back registrations. 3. **No Direct Hardware Serial Links:** Diagnostic values must currently be typed in manually rather than pulled directly from phoropter machines via serial cables.

## 5.5 Lessons Learnt

---

- **Domain-Specific Software Design:** General hospital software fails in eye care because optometry requires specialized optical grids rather than general text boxes.
- **Lightweight Architecture Wins:** For small-to-medium clinics, single-file SQLite databases offer faster performance, zero server maintenance, and simpler backups compared to heavy database clusters.
- **Workflow-First UX:** Keeping receptionists on the directory page upon registration significantly reduced front-desk friction.

## 5.6 Recommendations for Future Work

---

- **Direct Slit-Lamp USB Camera Capture:** Adding WebRTC camera capture to attach anterior segment photographs directly into the biomicroscopy tab.
- **Automated Cloud Backup Sync:** Adding scheduled cron uploads of `opticare.sqlite` to encrypted Google Drive or AWS S3 buckets.
- **National Health Insurance Scheme (NHIS) Tariff Integration:** Adding standard G-DRG claim form generation for accredited Ghanaian clinics.

## 5.7 Recommendations for Project Commercialization

---

1. **SaaS Subscription for Private Eye Practices:** Offer OptiCare V2 as an affordable cloud-hosted subscription (e.g., GHS 250/month) for private optometry clinics in Ghana. 2. **On-Premise Offline Bundles:** Package OptiCare V2 with a mini local server box and Wi-Fi router for rural clinics with unreliable internet connectivity. 3. **Optical Shop Add-On:** License the spectacle inventory and lens dispensing module to standalone optical shops.

## 5.8 References

---

1. American Academy of Ophthalmology. (2023). *Preferred Practice Pattern Guidelines: Comprehensive Eye Evaluation*. San Francisco: AAO. 2. World Health Organization. (2022). *World Report on Vision*. Geneva: WHO. 3. Somani, S., et al. (2021). "Implementation of electronic records in specialized eye care." *Journal of Medical Systems*, 45(4), pp. 42-51. 4. Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education. 5. Ministry of Health Ghana. (2021). *Standard Treatment Guidelines for Ophthalmic Conditions*. Accra: MOH Ghana.